1. Studi Kasus: BookStore E-Commerce

BookStore merupakan aplikasi e-commerce untuk penjualan buku secara online.

Sistem memiliki fitur:

- Mengelola data buku.
- Mengelola kategori buku.
- Mengelola penerbit.
- Mengelola pelanggan.
- Mencatat pesanan dan detail buku yang dibeli.
- Mencatat pembayaran.
- Mengelola pengiriman pesanan.

Mahasiswa diminta merancang database relasional yang memenuhi prinsip normalisasi, menjaga integritas data, dan dapat diimplementasikan menggunakan NestJS dan TypeORM.

2. Rancangan Tabel Database

Berikut rancangan tabel menggunakan nama berbahasa Inggris.

No.	Table	Fungsi
1	customers	Menyimpan data pelanggan
2	customer_addresses	Menyimpan alamat pelanggan
3	categories	Menyimpan kategori buku
4	publishers	Menyimpan data penerbit
5	books	Menyimpan informasi buku
6	authors	Menyimpan data penulis
7	book_authors	Menghubungkan buku dan penulis

8	orders	Menyimpan transaksi pesanan
9	order_items	Menyimpan detail buku dalam pesanan
10	payments	Menyimpan informasi pembayaran
11	shipments	Menyimpan informasi pengiriman

Tabel books akan menggunakan atribut dasar dari entity TypeORM yang Anda berikan.

---

3. Struktur Tabel books

Berdasarkan kode Anda, entity Book memiliki atribut berikut:

Column	Type	Keterangan
id	UUID	Primary key
title	VARCHAR(30)	Judul buku
author	VARCHAR(25)	Nama penulis
isbn	VARCHAR(45)	ISBN buku
publishedYear	INT	Tahun terbit, default 0
isAvailable	BOOLEAN	Status ketersediaan, default true
created_at	TIMESTAMP	Waktu data dibuat
updated_at	TIMESTAMP	Waktu data diperbarui
deleted_at	TIMESTAMP, nullable	Waktu soft delete

Ada dua penyesuaian yang saya rekomendasikan untuk konteks e-commerce:

1. Tambahkan price untuk menyimpan harga jual buku.
2. Tambahkan stock untuk menyimpan jumlah stok yang tersedia.

Dengan demikian, buku tidak hanya dapat dikelola sebagai data katalog, tetapi juga dapat dijual melalui sistem pesanan.

Berikut entity Book yang telah disesuaikan.

```
import {
  Column,
  CreateDateColumn,
  DeleteDateColumn,
  Entity,
  PrimaryGeneratedColumn,
  UpdateDateColumn,
} from 'typeorm';

@Entity({ name: 'books' })
export class Book {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'varchar', length: 30 })
  title: string;

  @Column({ type: 'varchar', length: 25 })
  author: string;

  @Column({ type: 'varchar', length: 45, unique: true })
  isbn: string;

  @Column({ type: 'int', default: 0 })
  publishedYear: number;

  @Column({ type: 'boolean', default: true })
  isAvailable: boolean;

  @Column({ type: 'decimal', precision: 12, scale: 2, default: 0
})
```

```
  price: number;

  @Column({ type: 'int', default: 0 })
  stock: number;

  @CreateDateColumn({ name: 'created_at' })
  createdAt: Date;

  @UpdateDateColumn({ name: 'updated_at' })
  updatedAt: Date;

  @DeleteDateColumn({ name: 'deleted_at', nullable: true })
  deletedAt: Date | null;
}
```

Catatan untuk mahasiswa:

- id menggunakan UUID sebagai primary key.
- isbn menggunakan constraint UNIQUE agar ISBN yang sama tidak tercatat lebih dari sekali.
- price menggunakan DECIMAL, bukan FLOAT, agar nilai harga tidak mengalami masalah presisi floating-point.
- stock digunakan untuk menyimpan jumlah buku yang tersedia dalam inventaris.
- isAvailable dapat menunjukkan apakah buku ditawarkan atau tersedia untuk dijual, tetapi status ini perlu disinkronkan dengan aturan stok.
- deletedAt digunakan untuk mendukung soft delete melalui TypeORM.

Panjang title 30 karakter dan author 25 karakter mengikuti entity awal Anda. Untuk aplikasi yang lebih realistis, panjang tersebut sebaiknya diperbesar karena judul buku dan nama penulis sering kali lebih panjang.

---

4. Struktur Relasi Database

Agar mahasiswa dapat mempelajari normalisasi, tabel-tabel lainnya dirancang sebagai berikut.

A. customers

Menyimpan data pelanggan.

Column	Type	Keterangan
id	UUID	Primary key
name	VARCHAR(150)	Nama pelanggan
email	VARCHAR(150)	Email pelanggan, unik
phone	VARCHAR(20)	Nomor telepon
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu diperbarui
deleted_at	TIMESTAMP, nullable	Waktu soft delete

Relasi:

customers (1) → (N) orders

Satu pelanggan dapat memiliki banyak pesanan.

B. customer_addresses

Menyimpan alamat pengiriman pelanggan.

Column	Type	Keterangan
id	UUID	Primary key
customer_id	UUID	Foreign key ke customers
recipient_name	VARCHAR(150)	Nama penerima
phone	VARCHAR(20)	Nomor telepon penerima
address	TEXT	Alamat lengkap
city	VARCHAR(100)	Kota
postal_code	VARCHAR(10)	Kode pos
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu diperbarui

Relasi:

customers (1) → (N) customer_addresses

Satu pelanggan dapat menyimpan beberapa alamat.

Untuk transaksi nyata, orders sebaiknya juga menyimpan snapshot alamat pengiriman saat checkout. Dengan begitu, perubahan alamat pelanggan tidak mengubah riwayat alamat pesanan lama.

C. categories

Menyimpan kategori buku.

Column	Type	Keterangan
id	UUID	Primary key
name	VARCHAR(100)	Nama kategori, unik
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu diperbarui

Contoh kategori:

- Programming
- Database
- Business
- Design
- Education

Relasi yang digunakan adalah one-to-many jika setiap buku hanya memiliki satu kategori utama.

Untuk menerapkan relasi tersebut, tambahkan category_id sebagai foreign key pada tabel books.

D. publishers

Menyimpan data penerbit.

Column	Type	Keterangan
id	UUID	Primary key
name	VARCHAR(150)	Nama penerbit
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu diperbarui

Relasi:

publishers (1) → (N) books

Satu penerbit dapat menerbitkan banyak buku.

Jika ingin menerapkan relasi ini, tambahkan publisher_id pada tabel books.

E. authors dan book_authors

Kedua tabel ini digunakan untuk mempelajari relasi many-to-many.

authors

Column	Type	Keterangan
id	UUID	Primary key
name	VARCHAR(150)	Nama penulis
created_at	TIMESTAMP	Waktu dibuat
updated_at	TIMESTAMP	Waktu diperbarui

book_authors

Column	Type	Keterangan
book_id	UUID	Foreign key ke books
author_id	UUID	Foreign key ke authors

Primary key dapat berupa gabungan (book_id, author_id).

Relasinya:

books (N) ↔ (N) authors

Penting: Jika menggunakan authors dan book_authors, kolom author pada entity Book sebaiknya dihapus. Menyimpan nama penulis langsung sekaligus mengelola relasi penulis melalui tabel terpisah akan menimbulkan redundansi dan berpotensi membuat data tidak konsisten.

Untuk latihan normalisasi, mahasiswa dapat diminta membandingkan kedua pendekatan tersebut.

---

5. Tabel Transaksi E-Commerce

A. orders

Menyimpan informasi utama pesanan.

Column	Type	Keterangan
id	UUID	Primary key
customer_id	UUID	Foreign key ke customers
order_number	VARCHAR(50)	Nomor pesanan, unik
status	VARCHAR(30)	Status pesanan
total_amount	DECIMAL(12,2)	Total pesanan
created_at	TIMESTAMP	Waktu pesanan dibuat
updated_at	TIMESTAMP	Waktu pesanan diperbarui

Contoh status:

- pending
- paid
- processing
- shipped
- completed
- cancelled

Relasi:

customers (1) → (N) orders

total_amount dapat disimpan sebagai snapshot nilai transaksi, tetapi perhitungannya harus dijaga agar konsisten dengan detail pesanan dan komponen biaya lainnya.

B. order_items

Menyimpan buku yang dibeli pada setiap pesanan.

Column	Type	Keterangan
id	UUID	Primary key
order_id	UUID	Foreign key ke orders
book_id	UUID	Foreign key ke books
quantity	INT	Jumlah buku
price	DECIMAL(12,2)	Harga satuan saat transaksi
subtotal	DECIMAL(12,2)	Subtotal item

Relasi:

- orders (1) → (N) order_items
- books (1) → (N) order_items

Mengapa price harus disimpan di order_items?

Misalnya:

- Harga buku saat ini: Rp150.000.
- Harga ketika pelanggan membeli buku minggu lalu: Rp130.000.

Riwayat transaksi harus tetap menunjukkan harga Rp130.000, walaupun harga katalog sekarang telah berubah.

Karena itu, order_items.price merupakan snapshot harga pada saat pembelian, bukan duplikasi yang harus selalu sama dengan books.price.

C. payments

Menyimpan informasi pembayaran.

Column	Type	Keterangan
id	UUID	Primary key
order_id	UUID	Foreign key ke orders
payment_method	VARCHAR(50)	Metode pembayaran
amount	DECIMAL(12,2)	Jumlah pembayaran
status	VARCHAR(30)	Status pembayaran
paid_at	TIMESTAMP, nullable	Waktu pembayaran berhasil
created_at	TIMESTAMP	Waktu dibuat

Untuk sistem sederhana, order_id dapat diberi constraint UNIQUE jika satu pesanan hanya boleh memiliki satu catatan pembayaran.

Jika sistem mendukung beberapa percobaan pembayaran, desainnya perlu memungkinkan satu pesanan memiliki beberapa catatan pembayaran.

D. shipments

Menyimpan informasi pengiriman.

Column	Type	Keterangan
id	UUID	Primary key
order_id	UUID	Foreign key ke orders
courier	VARCHAR(100)	Nama kurir
tracking_number	VARCHAR(100), nullable	Nomor resi
status	VARCHAR(30)	Status pengiriman
shipped_at	TIMESTAMP, nullable	Waktu pengiriman
delivered_at	TIMESTAMP, nullable	Waktu diterima

Untuk skenario sederhana, satu pesanan memiliki maksimal satu pengiriman sehingga order_id diberi constraint UNIQUE.

6. Ringkasan Relasi

Parent Table	Child Table	Relasi
customers	customer_addresses	One-to-Many
customers	orders	One-to-Many
categories	books	One-to-Many
publishers	books	One-to-Many
books	book_authors	One-to-Many
authors	book_authors	One-to-Many
orders	order_items	One-to-Many
books	order_items	One-to-Many
orders	payments	One-to-One, untuk rancangan sederhana
orders	shipments	One-to-One, untuk rancangan sederhana

---

7. ERD Konseptual

Berikut struktur relasi yang dapat dijadikan dasar tugas mahasiswa.

```
CUSTOMERS
   |
   | 1:N
   +---------------- CUSTOMER_ADDRESSES
   |
   | 1:N
   |
 ORDERS
   |
   | 1:N
   |
ORDER_ITEMS
   |
   | N:1
   |
  BOOKS
   |
   +-------------- CATEGORIES
   |
   +-------------- PUBLISHERS
   |
   | 1:N
   |
BOOK_AUTHORS
   |
   | N:1
   |
 AUTHORS

ORDERS
   |
   +-------------- PAYMENTS
   |
   +-------------- SHIPMENTS
```

Diagram ini merupakan rancangan konseptual. Pada ERD final, mahasiswa harus melengkapi seluruh atribut, primary key, foreign key, serta kardinalitas opsional.